

CIPHER (QVYRA)

Threat Model — Version 0.1 — June 2026

Sovereign Proof-of-Work Currency for Human and Autonomous-Agent Transactions

Status: Pre-implementation | Security posture: Unproven | Protocol: Conceptual

A sovereign protocol that cannot tolerate the truth about its own security is not sovereign.

1. Purpose

This document defines the principal security threats facing Cipher, a proposed sovereign proof-of-work Layer 1 blockchain designed for peer-to-peer transactions between humans and autonomous software agents.

Cipher is intended to operate without a foundation, pre-mine, founder allocation, protocol treasury, governance token, central issuer, central transaction authority, or custodian required for settlement. It also intends to provide proof-of-work issuance and consensus, privacy as a protocol baseline, direct wallet-to-wallet settlement, non-custodial Bitcoin to Cipher atomic swaps, structural resistance to exchange custody, and support for autonomous agent-to-agent payments.

This threat model does not assume that these properties have been achieved. It identifies what would need to be protected, who may attack it, where attacks may occur, and what remains unresolved. No property described in the whitepaper should be regarded as a security guarantee until it has been specified, implemented, independently reviewed and tested under adversarial conditions.

2. Security Principles

Distrust every participant. The protocol must assume that miners, users, agents, developers, counterparties, network peers and infrastructure providers may act maliciously.

Minimise trusted components. Security must not depend on a central server, foundation, approved wallet list, identity registry, trusted oracle or discretionary administrator.

Separate consensus from utility. Cipher issuance should not depend on proving that an external task was useful. A decentralised network can verify cryptographic work and protocol-valid state transitions. It cannot reliably determine whether off-chain economic activity was genuine or valuable.

Treat privacy as a system property. Transaction privacy must account for sender and receiver information, transaction amounts, output linkage, transaction graph analysis, timing, IP addresses, wallet fingerprints, agent identifiers, atomic-swap correlation, and external-service logs.

Avoid absolute claims. Cipher cannot truthfully claim to be impossible to stop, impossible to regulate, impossible to list on an exchange, completely anonymous, completely immutable, or free of governance. The defensible objective is to make censorship, surveillance, capture and unauthorised monetary change technically and economically difficult.

3. Assets to Protect

Monetary supply integrity — the network must preserve the intended issuance schedule and maximum supply. Threats include inflation bugs, duplicate issuance, malformed block rewards, integer overflows, invalid transaction acceptance, consensus divergence, and incorrect halving calculations. A failure may be irreversible.

Transaction correctness — only authorised holders can spend outputs, the same value cannot be spent twice, inputs and outputs balance, fees and rewards are calculated correctly, transactions cannot be altered after signing.

Consensus integrity — honest nodes must reach a consistent view of the valid chain, valid blocks, transaction ordering, accumulated proof of work, difficulty, block rewards, and chain reorganisations.

Private keys and signing authority — keys controlling Cipher and Bitcoin must remain confidential, including human wallet keys, agent wallet keys, mining-payout keys, swap-refund keys, and recovery secrets.

Transaction privacy — Cipher should protect sender identity, receiver identity, transferred amount, wallet balance, transaction history, linkage between outputs, and linkage between agents and their human operators.

Network privacy — the protocol should resist identification of transaction origin through IP address observation, peer timing, entry-node monitoring, traffic analysis, and global passive observation.

Atomic-swap safety — a Bitcoin to Cipher atomic swap must preserve atomicity, refundability, secret confidentiality, correct timelock ordering, resistance to transaction malleability, resistance to denial-of-service and griefing, and privacy across both chains.

Wallet-policy integrity — autonomous-agent wallets must spend only within authorised limits and must resist prompt injection, compromised plugins, malicious tools, policy manipulation, goal drift,

and unauthorised key export.

Software integrity — users must be able to distinguish genuine Cipher software from malicious binaries, compromised releases, dependency attacks, typosquatted packages, and forged wallet applications.

Protocol neutrality — no participant should gain undocumented privilege through hidden checkpoints, privileged keys, developer override mechanisms, trusted seed-node control, or covert inflation capability.

4. Adversaries

Opportunistic criminals — motivated by fund theft, wallet exploit, agent compromise. Capabilities include phishing, malware, botnets, fraudulent wallet software, and social engineering.

Sophisticated financial attackers — motivated by double-spending, market manipulation, and swap exploitation. Capabilities include significant capital, rented hash power, exchange access, and automated trading systems.

Dominant miners or mining pools — motivated by additional revenue, transaction censorship, and block reorganisation. Capabilities include substantial hash power, block-template control, and transaction-order control. Proof of work does not make miners automatically honest. It makes certain attacks costly.

State actors — motivated by user identification, transaction suppression, sanction enforcement, and infrastructure disruption. Capabilities include legal coercion, internet monitoring, traffic correlation, infrastructure seizure, and pressure on developers and hosting providers. A state actor may not need to defeat the cryptography. It may attack people, infrastructure and distribution channels.

Malicious or compromised developers — motivated by backdoor insertion, fund theft, and privileged upgrade paths. Capabilities include repository access, release-signing access, and influence over technical decisions. The absence of a foundation does not eliminate developer power.

Supply-chain attackers — motivated by compromising many users simultaneously. Capabilities include poisoning package registries, replacing binaries, and attacking build servers.

Malicious peers and network observers — motivated by deanonymisation and node isolation. Capabilities include operating many peers, controlling entry connections, and monitoring internet traffic.

Malicious atomic-swap counterparties — motivated by locking victim funds and exploiting timeout asymmetry. Capabilities include manipulating communication, delaying broadcasts, and monitoring both chains.

Malicious autonomous agents — motivated by stealing from other agents and generating fraudulent services. Capabilities include machine-speed interaction, unlimited identity generation, and automated negotiation.

Compromised autonomous agents — an otherwise legitimate agent may be compromised through prompt injection, malicious retrieved content, dependency compromise, or model manipulation. A compromised agent may possess valid signing authority and appear legitimate to the blockchain.

Custodians and exchanges — motivated by capturing trading volume and centralising liquidity. Capabilities include custom wallet infrastructure, interactive transaction services, and internal off-chain ledgers. No realistic protocol can assume that determined custodians are incapable of supporting interactive transactions.

5. Critical Attack Surfaces

Majority-Hash Attack

An attacker obtains enough hash power to exceed the honest network. On a young chain this may be inexpensive. The attacker may reorganise recent blocks, reverse payments, double-spend, censor transactions, or undermine confidence. Residual risk: no final proof-of-work algorithm has been selected. A new proof-of-work network may be cheap to attack regardless of its theoretical consensus design.

Inflation or Consensus Bug

A software defect permits creation of value beyond the emission schedule. A critical inflation bug may be discovered only after exploitation. Response options conflict with claims of immutability and absence of governance. Cipher has not specified how the community should respond to catastrophic consensus defects.

Agent Private-Key Compromise

An autonomous agent holds a spend-capable key in a runtime environment exposed to tools, prompts, plugins or network inputs. The blockchain cannot determine whether a validly signed agent payment reflects the human operator's true intent. This is one of Cipher's central unresolved problems.

Prompt-Injection Spending Attack

An agent reads malicious content instructing it to transfer Cipher to an attacker while disguising the instruction as part of a legitimate task. No model-based guardrail should be treated as a cryptographic control.

Undefined Privacy Architecture

The final privacy construction is undecided. Privacy as a baseline currently describes intent, not an implemented property. The specification must explicitly answer whether amounts are confidential, whether senders and receivers are hidden, whether transaction graphs are linkable, how double-spending is prevented privately, and how total supply is audited under confidential amounts.

Atomic-Swap Implementation Flaw

A cryptographic implementation flaw reveals a swap secret early or permits one party to claim without enabling the counterparty's claim. The swap construction has not yet been selected.

Cross-Chain Privacy Correlation

Observers correlate timing, amounts, wallet behaviour or revealed cryptographic material across Bitcoin and Cipher. Atomic swaps may create a strong public linkage between the two chains, potentially undermining Cipher's privacy objective.

Eclipse Attack

An attacker controls all peers connected to a target node. The victim sees a false or delayed view of the network, enabling double-spending facilitation, block delay, and censorship.

Repository or Release Compromise

An attacker gains control of the source repository or release process. Even without a foundation, maintainers become security-critical actors.

Custodial Exchange Implementation

An exchange implements interactive receive sessions and credits users in an internal database. Cipher cannot honestly promise that exchange custody is impossible.

6. Risk Register

Critical — Unresolved:

Early-chain majority-hash attack. Inflation or consensus bug. Undefined privacy architecture. Agent private-key compromise. Prompt-injection spending. Atomic-swap implementation flaw.

High — Unresolved:

Cross-chain privacy correlation. Mining centralisation. Difficulty manipulation. Network eclipse attack. Repository or release compromise. Interactive transaction denial-of-service. Fee-market and spam failure. Node-state exhaustion.

Cannot Be Eliminated:

Custodial exchange implementation. Wrapped or synthetic Cipher. Regulation of developers or miners. Global traffic correlation. Governance through maintainer influence.

7. Critical Unresolved Architectural Decisions

Cipher should not proceed to public-testnet claims until these decisions are documented:

Proof-of-work algorithm — SHA-256, memory-hard, Cuckoo Cycle, RandomX-like, or another construction. Security implications differ substantially.

Chain bootstrapping — initial difficulty, genesis construction, launch notice, early-miner advantage, rented-hash exposure, checkpoints, minimum chain work.

Transaction model — UTXO or account model, scripting, transaction aggregation, confidential amounts, pruning, output ownership.

Privacy mechanism — Mimblewimble-derived, confidential transactions, ring signatures, zero-knowledge proofs, stealth addressing, network privacy.

Supply schedule — block reward, halving interval, tail emission, smallest unit, fee-only security transition, justification for fixed supply.

Fee-market security — fee calculation, block capacity, minimum relay fee, spam controls, long-term miner compensation.

Software evolution — release process, upgrade activation, vulnerability handling, compatibility policy, contentious forks.

Atomic-swap protocol — hash-time-locked contracts, adaptor signatures, script requirements, refund model, liquidity discovery, privacy guarantees.

Agent-wallet authority — who holds keys, how spending is limited, how authority is revoked, how prompt injection is contained, whether human approval is required.

Custody resistance — exact interactive receive model, asynchronous receiving, wallet endpoint structure, trade-off between usability and custody friction.

8. Required Security Work Before Public Testnet

A formal protocol specification. A defined proof-of-work algorithm. An adversarial difficulty-adjustment analysis. A consensus-state machine. Transaction-format test vectors. Monetary-invariant tests. A peer-to-peer denial-of-service analysis. A privacy design with explicit

assumptions. A wallet key-management specification. An agent spending-authority model. An atomic-swap state machine. Failure and refund test cases. Reproducible builds. Repository and release-security procedures. Fuzz testing. Property-based testing. Chain-reorganisation simulations. Independent cryptographic review. Independent consensus review. Public disclosure of all known unresolved risks.

A public testnet is not evidence of security. It is an environment for discovering insecurity.

9. Residual Risks

Proof of work cannot guarantee decentralisation — mining may centralise through hardware, energy access, pools or economies of scale.

Privacy cannot be absolute — network metadata, endpoint compromise, counterparty information and cross-chain analysis may deanonymise users.

Self-custody permits irreversible loss — keys may be stolen, destroyed or lost.

Autonomous agents cannot prove intent cryptographically — a valid agent signature proves that a key authorised a transaction. It does not prove that the human beneficiary wanted it or that the agent understood it.

Custody cannot be prohibited completely — a third party can hold keys or issue claims representing Cipher regardless of the base protocol's intended use.

Software maintainers retain influence — removing formal governance does not remove social coordination, technical authority or emergency decision-making.

Governments can attack infrastructure and people — a protocol may resist transaction reversal while users, miners, developers and communication systems remain vulnerable to legal and physical coercion.

A small network is inherently vulnerable — before Cipher has substantial distributed hash power, node diversity and economic use, it may be inexpensive to disrupt.

10. Honest Security Statement

Cipher is not currently secure because Cipher does not yet exist as a complete protocol.

Its intended properties are ambitious and internally demanding: proof-of-work security without concentrated mining; privacy without unverifiable supply; autonomous agent spending without

uncontrolled key risk; interactive transactions without unacceptable availability loss; atomic swaps without cross-chain deanonymisation; resistance to custody without pretending custody can be prohibited; minimal governance without denying the existence of developer and miner influence.

The purpose of this threat model is not to claim that these tensions have been solved. It is to prevent the project from hiding them.

Cipher should be considered successful only if independent participants can verify its rules, challenge its assumptions, reproduce its software, identify its failure modes and operate without trusting a privileged organisation.

No one should trust Cipher because of its whitepaper, its philosophy or its founders.

They should trust only what they can verify.